

SEW-Offload: Static Expert-Window Scheduling for Prefetch-Hidden MoE Offloading on Ascend NPUs

Abstract

Mixture-of-Experts (MoE) inference stresses device memory because only a small subset of experts is activated per token, while the full expert parameter set still has to be stored somewhere. Existing MoE offloading systems are largely designed around GPU assumptions: expert caching, host-to-device prefetching, and overlap through CUDA streams. These assumptions become incomplete on Ascend NPUs. Ascend execution is more sensitive to stable tensor addresses, fixed shapes, graph replay, and explicitly scheduled data movement. This document describes the design of SEW-Offload, a static expert-window scheduling system for HBM-limited MoE inference on Ascend NPUs. SEW-Offload does not retrain the router, does not alter top-k expert activation, and does not drop tokens. Instead, it reinterprets expert offloading as a scheduling problem: fixed HBM expert slots provide stable execution entries, deadline-aware prefetching decides which missing experts should be loaded before they are needed, and hit-first phased execution hides the remaining miss latency behind already available expert computation. The goal is to minimize exposed prefetch stall, not merely to maximize cache hit rate.

1 Motivation

Large MoE models such as Qwen3-30B-A3B activate only a small number of experts for each token, but their total expert weights can still exceed the HBM budget available to a single NPU card, especially when KV cache, long-context serving, and concurrent requests compete for memory. Expert offloading is therefore a natural way to serve larger MoE models under limited HBM. The central question is not simply whether expert weights can be moved between host memory and HBM, but whether the movement cost can be hidden well enough to preserve serving latency and throughput.

GPU-oriented MoE offloading typically relies on a dynamic expert cache: keep some experts resident, predict future experts, prefetch missing experts, and overlap host-to-device copies with GPU computation. This abstraction is useful but under-specifies the control surface exposed by Ascend NPUs. Ascend runtime and kernels benefit from stable tensor addresses, fixed execution shapes, graph capture and replay, and explicit memory movement orchestration. If an offloaded expert is loaded into arbitrary weight tensors at runtime, the device may lose the very regularity that makes Ascend execution efficient.

SEW-Offload starts from the observation that vLLM Ascend already provides a grouped MoE execution representation: token-level routing results are converted into per-expert token counts or group lists before grouped matrix multiplication. Therefore, SEW-Offload must not claim “per-token to per-expert count conversion” as its contribution. The missing piece is an offloading-oriented execution window for expert weights: only a fixed number of expert slots are present in HBM, each slot has a stable tensor address, and the runtime dynamically maps active expert ids to those slots while orchestrating loads, prefetches, replacements, and phased grouped execution.

2 Problem Statement

Given an MoE layer with a set of experts E , a limited HBM budget that can hold only $S \ll |E|$ expert slots, and a stream of inference steps that produce dynamic active expert sets, SEW-Offload aims to execute the original MoE semantics while minimizing exposed offloading stall.

The system must preserve three correctness constraints:

- The router logits, top-k expert ids, and gate weights are unchanged.
- Every token is dispatched to the same experts as in the non-offloaded baseline.
- Expert outputs are combined with the same combine semantics as the existing vLLM Ascend grouped MoE path.

The performance objective is not raw cache hit rate. A miss is harmful only when its loading time is exposed on the critical path. For a layer or execution phase, the exposed stall can be written as:

$$T_{\text{stall}} = \max(0, T_{\text{load-miss}} - T_{\text{overlap}}), \quad (1)$$

where $T_{\text{load-miss}}$ is the host-to-HBM time required by missing experts and T_{overlap} is the amount of useful routing, dispatch, resident-expert computation, later-layer computation, or cross-step computation that can run while those experts are being loaded. SEW-Offload is designed to increase T_{overlap} and reduce the miss load that remains on the critical path.

3 Old Assumptions and New Control Surface

Table 1 summarizes the design shift.

Aspect	GPU-style assumption	Ascend-aware redesign
Expert residency	Dynamic expert cache with flexible weight objects	Fixed HBM expert slots with stable tensor addresses
Execution regularity	Dynamic kernels and stream overlap are often acceptable	Prefer fixed shapes, stable entries, graph replay, and few graph variants
Offloading objective	Reduce miss count or copy time	Reduce exposed stall after overlap
Miss handling	Wait for missing experts, or prefetch predicted experts	Execute hit experts first while miss experts are loaded
Dispatch representation	Often discussed from token-level routing	Reuse existing per-expert count/grouped execution; redesign weight residency
Data movement	Mostly host-to-device copy scheduling	Coordinate host-to-HBM loading with NPU-side movement and grouped compute

Table 1: SEW-Offload changes the optimization target from dynamic expert cache management to static expert-window scheduling with prefetch-hidden execution.

4 Design Overview

SEW-Offload consists of three coupled mechanisms.

4.1 Static Expert Slots

The HBM budget is represented as a fixed slot pool:

$$\mathcal{S} = \{s_0, s_1, \dots, s_{S-1}\}.$$

Each slot owns preallocated weight tensors for the expert MLP weights. The address, layout, dtype, and capacity of each slot are stable across inference steps. The runtime maintains two mappings:

$$\text{expert_to_slot} : E \rightarrow \mathcal{S} \cup \{\perp\}, \quad \text{slot_to_expert} : \mathcal{S} \rightarrow E \cup \{\perp\}.$$

Stable slots separate the dynamic expert identity from the static device-side execution entry. The grouped MoE backend sees slot-local weights; the runtime ensures that each slot contains the correct expert before it is used. This design supports Ascend graph/static-kernel friendliness because the device-side weight entry points are fixed even when the logical experts change.

4.2 Deadline-Aware Prefetch and Orchestration

The prefetch planner treats every candidate expert load as a job with a deadline. For an expert e , the planner estimates:

- the next expected use time or layer/step deadline $d(e)$;
- the loading cost $c(e)$ from host memory into an HBM slot;
- the execution value $v(e)$, derived from expected token count, recent locality, and miss penalty;
- the replacement cost of evicting the current slot owner.

The planner prioritizes experts that are likely to be used soon, have high token counts, and would cause large exposed stall if missing. Unlike a pure LRU cache, the planner is allowed to choose a lower hit-rate schedule if that schedule hides more loading time behind useful computation.

The most important prefetch opportunities are:

- **Intra-layer overlap:** load miss experts while already resident experts execute.
- **Inter-layer overlap:** use active experts from nearby layers and previous observations to prefetch before the next MoE layer is reached.
- **Cross-step decode overlap:** use decode-step locality to prefetch experts for the next token step while later layers of the current step are still executing.

4.3 Hit-First Phased Execution

When the current MoE layer produces an active expert set A , SEW-Offload partitions it into resident hits and missing experts:

$$A_{\text{hit}} = \{e \in A \mid \text{expert_to_slot}(e) \neq \perp\}, \quad A_{\text{miss}} = A \setminus A_{\text{hit}}.$$

Instead of blocking until all misses are loaded, SEW-Offload executes available experts first. The layer is decomposed into a small number of grouped execution phases:

1. immediately execute a grouped MLP phase for A_{hit} ;
2. concurrently load selected experts from A_{miss} into available or replaceable slots;
3. execute one or more follow-up grouped MLP phases as miss experts become ready;
4. combine all partial expert outputs according to the original token and gate mapping.

This mechanism converts a full-layer blocking miss into a phased execution problem. It also avoids per-expert tiny kernels: ready experts are still batched into grouped MLP phases, preserving the execution style of the existing Ascend MoE backend.

Component	Responsibility
ExpertStore	Owns the complete expert weights in host memory and exposes expert-level or tile-level load interfaces.
SlotPool SlotManager	Preallocates fixed HBM expert slots and provides stable weight tensor entries. Maintains expert-to-slot mappings, handles admission, eviction, and slot validity checks.
WorkingSetObserver	Consumes existing vLLM Ascend routing outputs, especially per-expert token counts and active expert ids, without changing router behavior.
PrefetchPlanner	Scores candidate experts using deadline, token count, locality, and load cost; emits asynchronous prefetch jobs.
PhaseScheduler	Builds hit-first grouped execution phases and decides when to wait for prefetched miss experts.
ExecutionAdapter	Connects slot-local weights to the existing grouped MoE execution boundary with minimal intrusion.
Telemetry	Records hit rate, miss load time, exposed stall, copy/compute overlap, slot churn, and correctness checks.

Table 2: SEW-Offload runtime components.

5 System Components

6 Scheduling Algorithm

At each MoE layer, the runtime performs the following steps:

Input:

active experts A from existing grouped MoE metadata
per-expert token counts C
fixed slot pool S
pending prefetch jobs P

1. Refresh slot readiness for completed prefetch jobs.
2. Partition A into hit experts A_hit and miss experts A_miss.
3. Launch or continue prefetch jobs for high-value miss experts.
4. Build phase 0 from A_hit and execute it immediately.
5. While unresolved experts remain:
 - a. admit newly ready experts into a follow-up phase;
 - b. if no ready expert exists, wait for the earliest useful prefetch;
 - c. execute the ready phase through grouped MLP;
6. Combine phase outputs according to original routing and gate weights.
7. Use current observations to plan prefetches for future layers or steps.

The planner uses a score such as:

$$\text{score}(e) = \alpha \cdot \widehat{C}(e) + \beta \cdot \text{locality}(e) + \gamma \cdot \text{deadline_urgency}(e) - \delta \cdot \text{load_cost}(e) - \eta \cdot \text{eviction_cost}(e), \quad (2)$$

where $\widehat{C}(e)$ is the predicted or observed token count for expert e . The exact score is a policy choice. The key point is that admission is not optimized for hit rate alone; it is optimized for reducing the stall that survives overlap.

7 Ascend-Specific Design Rationale

SEW-Offload is designed around Ascend-specific execution properties rather than treating the NPU as a GPU replacement.

Stable weight addresses. Fixed slots allow graph capture and static-kernel paths to observe stable device-side weight entries. Logical expert ids may change, but the physical slot tensors do not.

Fixed execution windows. The number of HBM slots is fixed, and the runtime can restrict execution to a small set of slot-window variants. This reduces dynamic graph diversity compared with arbitrary expert weight objects.

Explicit movement orchestration. Ascend systems expose data movement as a first-class optimization surface. SEW-Offload separates host-to-HBM expert loading from NPU-side grouped compute and records whether the copy time is hidden or exposed.

Avoiding small expert kernels. Hit-first execution does not mean one kernel per expert. It groups ready experts into phases so that the backend can continue using grouped matrix multiplication.

8 Integration with vLLM Ascend

The intended implementation is low-intrusion. SEW-Offload should live in a new package, for example `vllm_ascend/moe_offload/`, and remain disabled by default. The first integration point is the expert execution boundary in the existing fused MoE path, after routing and grouped metadata are available and before the grouped expert MLP consumes weights.

The initial MVPs should be staged as follows:

1. **Observer mode:** record active experts, per-expert token counts, layer locality, and estimated reuse without changing execution.
2. **Synchronous slot mode:** load missing experts into fixed slots synchronously to validate correctness under artificial slot budgets.
3. **Prefetch mode:** add asynchronous prefetch jobs and measure exposed stall reduction.
4. **Hit-first phased mode:** split layer execution into resident and ready-miss phases, while preserving grouped MLP execution.
5. **Graph/static-kernel mode:** evaluate whether fixed slot windows improve graph replay stability and reduce overhead under selected capacity tiers.

9 Correctness and Failure Handling

Correctness is protected by treating offloading as a weight-residency transformation rather than a routing transformation. The runtime must verify that every slot used in a phase contains the expected expert. A practical implementation should maintain expert-id checksums or version counters for slots, assert that phase-local expert ids match the original routed ids, and compare per-layer outputs against a non-offloaded baseline during testing.

Failure handling should be conservative. If prefetch fails or the planner cannot admit an expert before its deadline, the runtime falls back to a synchronous load for that expert. This may hurt latency but preserves semantics. If slot pressure is too high, the system should expose the stall in telemetry rather than silently dropping experts or changing router behavior.

10 Evaluation Plan

The evaluation should answer four questions.

1. **Does fixed slotting preserve correctness?** Compare layer outputs and end-to-end generated tokens against the non-offloaded baseline.
2. **Does orchestration reduce exposed stall?** Measure total host-to-HBM load time, hidden load time, and exposed stall separately.

3. **Does hit-first phased execution improve latency under misses?** Compare blocking miss execution with phased execution under the same slot budget and workload.
4. **Does Ascend-specific slot stability help graph/static execution?** Compare dynamic expert cache, fixed slots without graph replay, and fixed slots with selected graph/static-kernel variants.

Key metrics include time-to-first-token, time-per-output-token, throughput, P50/P95/P99 latency, slot hit rate, exposed miss stall, copy/compute overlap, MTE/Cube overlap where available, slot churn, and HBM savings.

11 Expected Contributions

SEW-Offload is expected to contribute:

- an Ascend-aware reformulation of MoE offloading as static expert-window scheduling rather than dynamic expert caching alone;
- a fixed expert-slot abstraction that preserves stable HBM weight entries under dynamic expert identities;
- a deadline-aware prefetch planner that optimizes exposed stall instead of cache hit rate alone;
- a hit-first phased execution strategy that hides miss loading behind resident expert computation while retaining grouped MLP execution;
- a low-intrusion integration plan for vLLM Ascend that reuses existing per-expert count/grouped MoE metadata and does not modify router semantics.

12 Limitations

The current design assumes that host-to-HBM expert loading can be initiated asynchronously and that the runtime can safely bind loaded weights into fixed HBM slots before grouped execution. The design also assumes useful locality exists in decode or continuous batching workloads; prefill-heavy workloads with weak expert reuse may expose more load time. Finally, graph/static-kernel benefits must be validated empirically on Ascend hardware, because dynamic token counts may still create graph variants even when weight slots are stable.